

Построение пайплайнов данных

Оркестрация пайплайнов

Малахов Дмитрий Андреевич

Новосельцев Максим Сергеевич

23 января 2026 года

Кто и где использует ETL/ELT

- **Данных стало слишком много**, поэтому аналитические запросы работают долго, а за хранение лишних данных приходится платить.
- **Данные лежат в разных местах**, поэтому аналитики не могут с ними работать.

ETL-процесс

1. Подключение к источнику
2. Выгрузка данных из источника
3. Первичная очистка данных
4. Маппинг данных
5. Агрегация данных
6. Загрузка данных в систему-приёмник

Как реализовать ETL/ELT-процесс

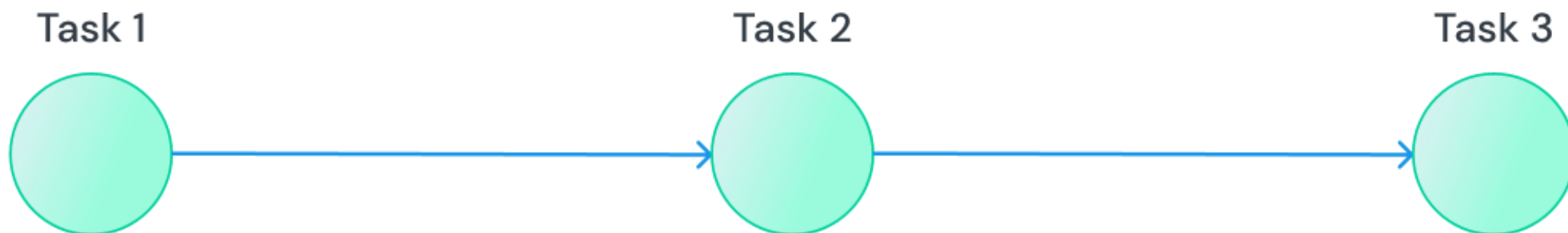
1. Понять задачу
2. Получить доступ к данным
3. Проверить полученные данные
4. Написать код ETL/ELT-процесса
5. Запустить автоматическое исполнение кода

CRON

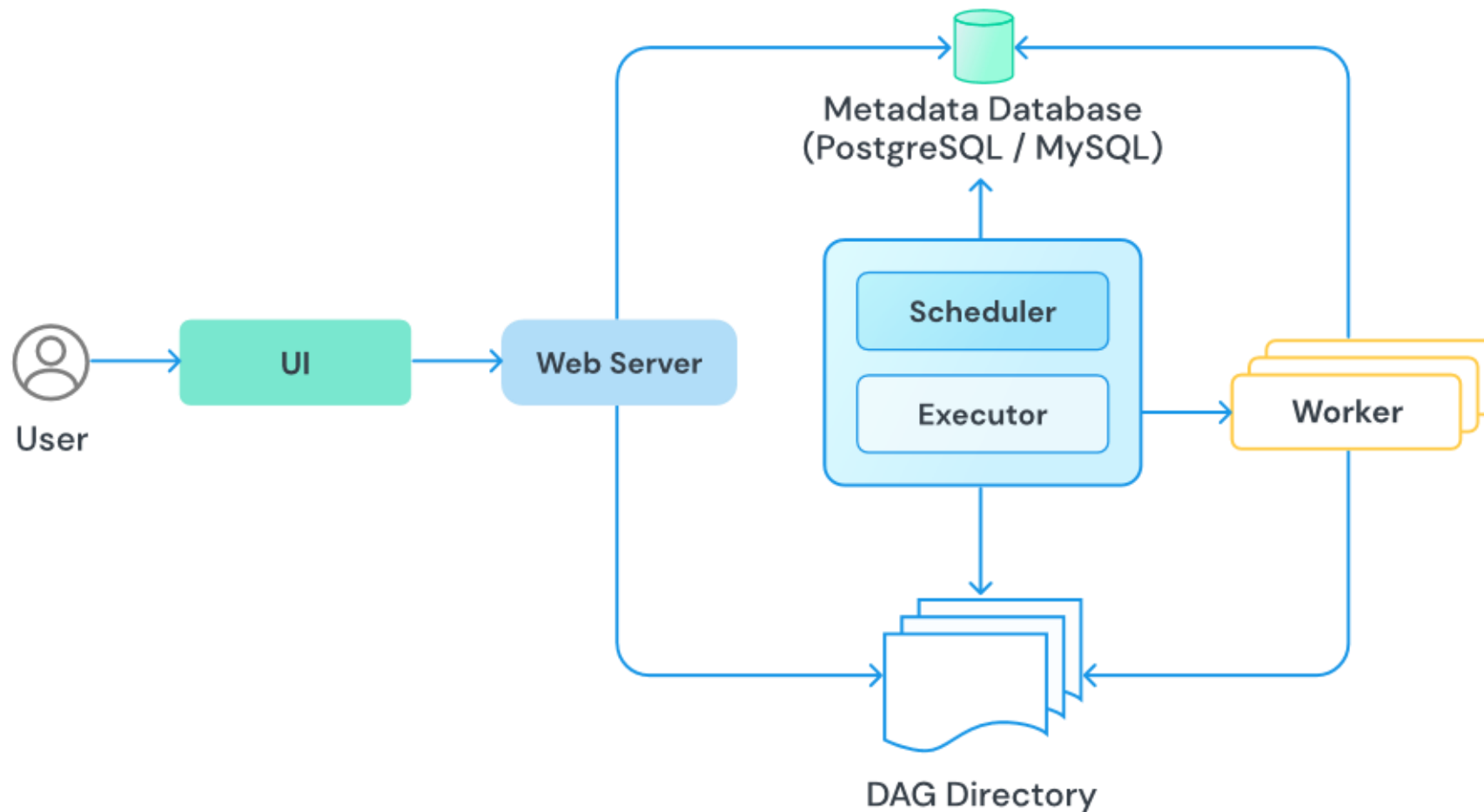
```
GNU nano 2.2.6      File: /tmp/crontab.ebLDpV/crontab      Modified
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
[ Read 22 lines ]
^G Get Help  ^O WriteOut  ^R Read File  ^Y Prev Page  ^K Cut Text   ^C Cur Pos
^X Exit      ^J Justify   ^W Where Is  ^V Next Page  ^U UnCut Text ^T To Spell
```

Что такое DAG?

DAG — это ориентированный ациклический граф, концептуальное представление серии действий или, другими словами, математическая абстракция конвейера данных (data pipeline).



Компоненты архитектуры



Жизненный цикл

1. Parsing
2. Scheduling
3. Queuing
4. Execution
5. Completion

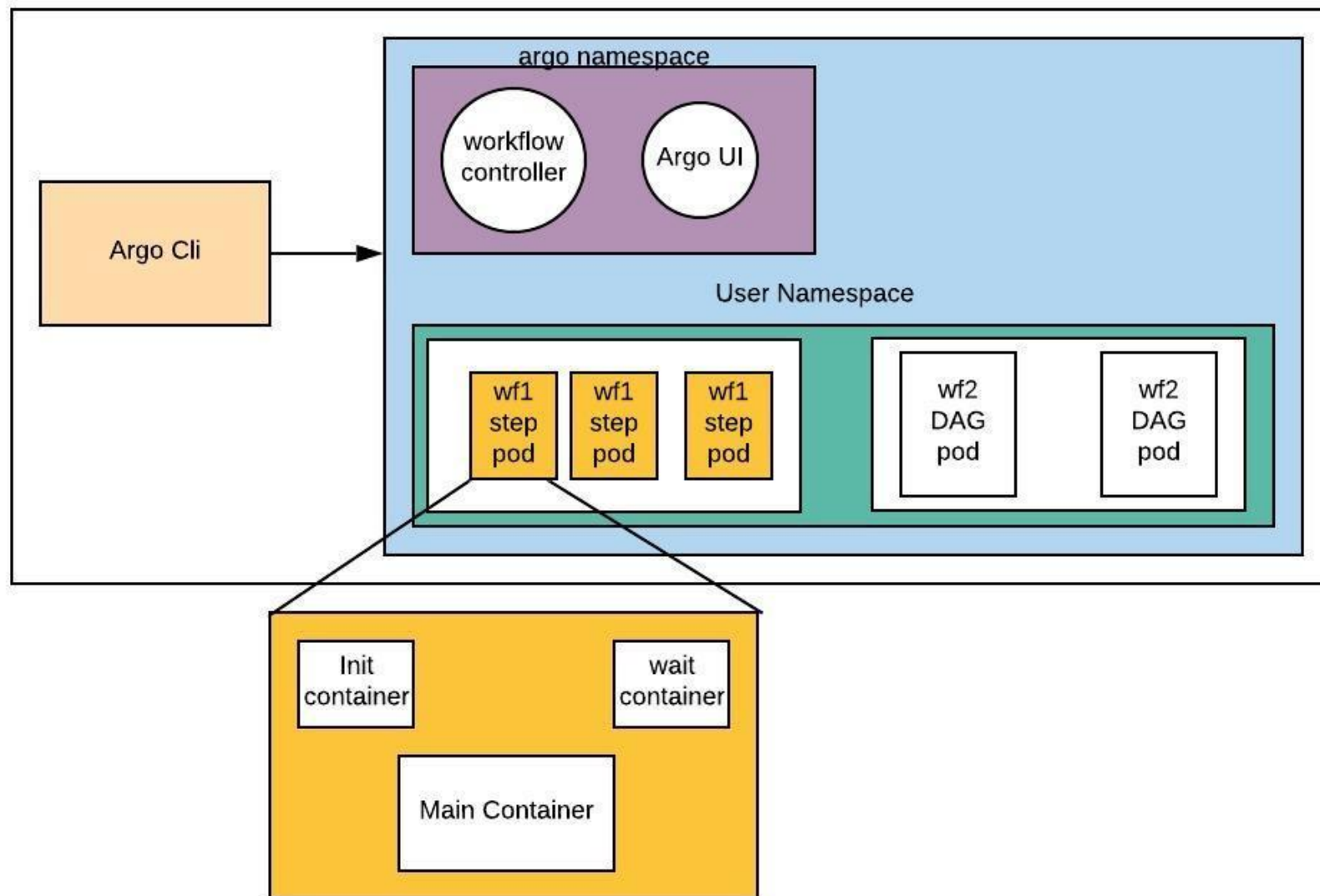
deferred failed queued removed restarting running scheduled shutdown skipped success up_for_reschedule up_for_retry upstream_failed no_status

Характеристика	BashOperator	PythonOperator
Когда использовать	Системные команды, перемещение файлов, запуск бинарников	Сложная логика, работа с API, обработка данных в Pandas
Скорость запуска	Почти мгновенно	Зависит от объема импортируемых библиотек
Читаемость	Хорошая для коротких однострочников	Отличная для разработчиков Python
Передача данных	Через временные файлы или переменные окружения	Нативная через механизм XCom

Проблемы Airflow

1. Масштабируемость
2. Отсутствие Kubernetes-нативности
3. Слабый GitOps

ARGO Workflow Overview



JSON/YAML

▶ METADATA

▶ SPEC

```
1  metadata:
2    name: wonderful-tiger
3    namespace: argo
4    labels:
5      example: "true"
6  spec:
7    workflowMetadata:
8      labels:
9        example: "true"
10   entrypoint: argosay
11   arguments:
12     parameters:
13       - name: message
14         value: hello argo
15   templates:
16     - name: argosay
17       inputs:
18         parameters:
19           - name: message
20             value: "{{workflow.parameters.message}}"
21       container:
22         name: main
23         image: argoproj/argosay:v2
24         command:
25           - /argosay
26         args:
27           - echo
28           - "{{inputs.parameters.message}}"
29   ttlStrategy:
30     secondsAfterCompletion: 300
31   podGC:
32     strategy: OnPodCompletion
33
```

i Basic completion for YAML. Switch to JSON for full auto-completion. [Learn how to get auto-completion in your IDE.](#)

You can find manifests [in the examples](#) or templates in [Workflow Template Catalog](#).

```
≡ Send Snippet...
> kubectl get pods -n argo
NAME                                READY   STATUS             RESTARTS   AGE
argo-server-d76f4d4b7-wgxz9        1/1     Running            0          2m28s
httpbin-f5ccc9c6-bs544            1/1     Running            0          17m
lovely-tiger-xjls1                0/2     PodInitializing    0          30s
minio-5877d79784-xrjs8            1/1     Running            0          17m
workflow-controller-85d57f9fc7-srg55 1/1     Running            0          17m
> kubectl get pods -n argo
NAME                                READY   STATUS             RESTARTS   AGE
argo-server-d76f4d4b7-wgxz9        1/1     Running            0          2m46s
httpbin-f5ccc9c6-bs544            1/1     Running            0          18m
lovely-tiger-xjls1                0/2     Completed          0          48s
minio-5877d79784-xrjs8            1/1     Running            0          18m
workflow-controller-85d57f9fc7-srg55 1/1     Running            0          18m
> kubectl get logs lovely-tiger-xjls1 -n argo
```

spec:

ttlStrategy: secondsAfterCompletion: 300 # Удалить через 5 минут после завершения

secondsAfterSuccess: 60 # Удалить через 1 минуту после успеха

secondsAfterFailure: 3600 # Удалить через 1 час после сбоя

```

1  apiVersion: argoproj.io/v1alpha1
2  kind: WorkflowTemplate
3  metadata:
4    name: log-first-template
5  spec:
6    templates:
7    - name: log-first
8      container:
9        image: alpine:latest
10       command: ["sh", "-c"]
11       args:
12         - |
13           echo "===== ПЕРВЫЙ ШАБЛОН ====="
14           echo "Сообщение от первого логера"
15           echo "Текущее время: $(date)"
16           echo "=====+====="
17

```

```

1  apiVersion: argoproj.io/v1alpha1
2  kind: WorkflowTemplate
3  metadata:
4    name: log-second-template
5  spec:
6    templates:
7    - name: log-second
8      container:
9        image: alpine:latest
10       command: ["sh", "-c"]
11       args:
12         - |
13           echo "===== ВТОРОЙ ШАБЛОН ====="
14           echo "Сообщение от второго логера"
15           echo "Хост: $(hostname)"
16           echo "=====+====="

```

```

1  apiVersion: argoproj.io/v1alpha1
2  kind: Workflow
3  metadata:
4    generateName: combined-workflow-
5  spec:
6    entrypoint: main-steps
7    templates:
8
9    - name: main-steps
10      steps:
11        - - name: call-first
12            templateRef:
13              name: log-first-template
14              template: log-first
15
16        - - name: call-second
17            templateRef:
18              name: log-second-template
19              template: log-second

```

Домашнее задание

Разработайте 3 и более универсальных WorkflowTemplate, которые решают частые задачи оркестрации (на ваш выбор — что чаще всего приходится повторять).

Для каждого шаблона:

- Определите, какие входные параметры (inputs.parameters) нужны, чтобы он был гибким и переиспользуемым
- Продумайте, что он возвращает (outputs.parameters или outputs.artifacts)
- Напишите YAML-код шаблона

Затем создайте основной Workflow, который вызывает ваши шаблоны и собирает их в осмысленный пайплайн.